

AI-Recruiter — итоговый отчёт

1. Постановка задачи

Классический поиск по ключевым словам не понимает контекста и синонимов: кандидат, написавший «разрабатывал архитектуру распределённых систем на Go», может не попасть в выдачу по жёсткому тегу «Backend Engineer». Рекрутеру нужен не просто отсортированный список, а текстовое обоснование, почему конкретный человек подходит под сложные требования вакансии.

Цель — мультиагентная система: загружаем вакансию → векторный поиск отбирает топ-N кандидатов из базы резюме → команда LLM-агентов (Технический Скринер и HR-Аналитик) проводит построчный аудит → генерируется карточка с процентом соответствия, плюсами/минусами/рисками и вердиктом «Рекомендован / В резерв / Отказ».

2. Описание решения

Конвейер состоит из модулей:

- Парсинг (parsing) — извлечение текста из PDF/DOCX/ODT/TXT.
 - Чанкинг (chunking) — нарезка резюме по секциям (опыт, навыки, образование и др.).
 - Эмбединги и векторный поиск (embeddings) — индексация чанков в ChromaDB с fallback на in-memory хранилище.
 - Тулы (tools) — `semantic_search`, `rank_candidates`, `extract_experience_years`, `check_mandatory_skills`, `get_resume_chunks`.
 - Агенты (agents) — Технический Скринер (стек и хард-скиллы) и HR-Аналитик (карьерная динамика, red flags); fallback — rule-based скоринг без LLM.
 - Отчёт (report) — агрегация взвешенного скорпа, вердикт, карточка в Markdown/HTML.
- Эмбединги — sentence-transformers (intfloat/multilingual-e5-large) с префиксами e5; при отсутствии модели используется детерминированный хэшинг-эмбеддер на n-граммах. LLM — Ollama или OpenAI-совместимый API с переключением на rule-based fallback.

3. Результаты экспериментов

- MRR: 0.900
- Hit@1: 0.800
- Hit@3: 1.000
- Hit@5: 1.000
- Precision@5: 0.760

Разбивка по вакансиям

- Backend-разработчик Python (Middle+) [Backend-разработчик (Python)] — MRR 1.000, топ роли: Backend-разработчик (Python), Backend-разработчик (Python), Backend-разработчик (Python)
- Senior DevOps-инженер [DevOps-инженер] — MRR 1.000, топ роли: DevOps-инженер, DevOps-инженер, DevOps-инженер
- QA Automation Engineer (Java/Python) [QA-инженер] — MRR 1.000, топ роли: QA-инженер, Backend-разработчик (Python), Frontend-разработчик (React)
- ML-инженер (NLP/LLM) [ML-инженер] — MRR 1.000, топ роли: ML-инженер, ML-инженер, ML-инженер
- Data Analyst (Junior+) [Data Analyst] — MRR 0.500, топ роли: Backend-разработчик (Python), Data Analyst, Data Analyst

4. Выводы

На демо-данных (100 синтетических резюме, 5 вакансий) семантический поиск стабильно поднимает релевантных кандидатов наверх: $MRR \approx 0.9$, $Hit@3 = 1.0$. Это подтверждает, что

даже лексический fallback-эмбеддер даёт осмысленное ранжирование, а с полноценной эмбеддинг-моделью результат должен ещё улучшиться.

Ограничения: метрики посчитаны на синтетических данных с известной ролью кандидата; агенты работают в rule-based режиме (LLM не вызывался). Дальнейшие шаги — подключить sentence-transformers + ChromaDB, реальный LLM для генерации объяснений и расширить демо-базу.